AI ENGINEERING PROJECTS by Jyothi Prakash Kalmanje V2

Architect's build guide — specifications, performance models, acceptance criteria
Every tuning range is marked. Nothing is asserted that cannot be derived or cited.

WHAT AI ENGINEERING IS, AND WHY A PORTFOLIO IS THE PROOF

start here — the discipline, how it differs from adjacent ones, and what a reviewer is actually looking for

The definition

AI engineering is **building reliable software on top of models you did not train**. The model is a component — probabilistic, externally versioned, expensive per call — and the work is making a dependable system out of it.

How it differs from its neighbours

Discipline	Central question
ML research	Can a model learn this at all? Produces architectures and papers.
ML engineering	How do we train, evaluate and serve *our own* model at scale? Features, pipelines, drift.
AI engineering	How do we build a product on an existing model, and know when it is wrong? Retrieval, tools, guardrails, evaluation, cost.
Software engineering	Everything the above still needs: APIs, state, deployment, observability.

The distinction that matters in interviews. You are not expected to invent architectures. You are expected to make a non-deterministic component behave predictably enough to put in front of users — and to prove it with numbers rather than demos.

What the work actually is

- **Retrieval and context** — getting the right information into the prompt
- **Orchestration** — tools, loops, and when to stop
- **Evaluation** — the gate that makes iteration safe
- **Guardrails and security** — injection, leakage, excessive agency
- **Cost and latency** — engineering constraints, not afterthoughts
- **Operations** — tracing, versioning, rollback, on-call

Why a portfolio, and not a certificate

The field moves faster than curricula, and almost everyone can now produce a working demo. That makes demos worthless as evidence — and makes the parts nobody demos the only useful signal.

What everyone has	What almost nobody has
A notebook that answers questions over a PDF	An evaluation set, a baseline, and a measured improvement over it
A screenshot of a good answer	The failure cases, documented, with what was done about them
"It uses RAG"	Retrieval recall measured separately from generation faithfulness
A deployed URL	Traces, latency percentiles and cost per request
A tutorial's architecture	A decision record explaining what was rejected, and why
A repository	A repository someone else can run in one command

The uncomfortable part. A reviewer spends a few minutes on a repository. They are not reading your model code — they are looking for whether you know how to tell if it works. Evaluation is the fastest signal of seniority in this field, and its absence is the fastest signal of the opposite.

How to use the projects on this sheet

- **Depth over breadth.** Two projects taken to production quality beat six demos, every time.
- **Each one targets a distinct skill** — retrieval, self-hosting, observability, training, real-time. They are not variations of the same build.
- **Build them in order.** Later projects assume the evaluation and observability habits established earlier.
- **Measure before and after every change.** An improvement you cannot demonstrate is a preference.
- **Write down what failed.** The failure log is the most convincing document in the repository.

Evaluation is the spine. It is the difference between changing a prompt and *improving* a system. Every project here is built around it, and it is the section reviewers reach for first.

What this is not

- **Not a course.** No sequence of videos substitutes for shipping something and watching it fail.
- **Not model training.** One project touches fine-tuning; the discipline is mostly about systems around models.
- **Not framework knowledge.** Frameworks change yearly; retrieval quality, evaluation and cost control do not.

The test to apply to your own work. Can you state, with a number, how good your system is — and how you would know if a change made it worse? If not, that is the next thing to build, whatever else is on the roadmap.

PORTFOLIO ARCHITECTURE

five projects, two systems — build them as a composed whole

SYSTEM A — the retrieval platform (Projects 1 + 3 + 4)

P1 · RAG service — hybrid retrieval + rerank, citation-enforced generation
→ **P3 · Observability** — traces, metrics, cost, CI regression gate
→ **P4 · Fine-tuned extractor** — structured output for the same domain corpus

One golden evaluation set serves all three
P1 uses it to measure retrieval and faithfulness. P3 runs it in CI as the regression gate.
P4 reuses its domain and held-out split to prove the fine-tuned model beats the prompted baseline.
This shared artefact is what turns three projects into one coherent engineering story.

SYSTEM B — the latency-constrained runtime (Projects 2 + 5)

P2 · Local inference service — quantized SLM, measured throughput, schema-validated output
→ **P5 · Real-time voice pipeline** — ASR → reasoning → TTS under a budget, with degradation and replay

P2 becomes the reasoning stage of P5
A local model is the natural choice for the voice loop: no network hop, so the LLM stage of your latency budget is bounded by hardware you control and can measure directly.
The benchmark numbers from P2 become the budget line items in P5.

CAPABILITY MATRIX — what each project actually evidences

● primary evidence ○ partial · incidental

Competency	P1 RAG	P2 Local	P3 Obs	P4 Tune	P5 RT	Hiring signal
Distributed system design	●	○	●	·	●	High
Measurement & evaluation rigour	●	●	●	●	●	Highest
Performance / cost engineering	○	●	●	○	●	High
Model training & alignment	·	·	·	●	·	Specialist
Production operations / SRE	○	○	●	·	●	Scarcest

SEQUENCING

order and dependency

#	Project	Gate before moving on
1	**P1 · RAG** phases 1–2	End-to-end answers with citations traceable to source
2	**P1** phase 3 — golden set	Baseline retrieval and faithfulness numbers recorded
3	**P3 · Observability**	Every request traceable; CI fails on a seeded regression
4	**P2 · Local SLM**	Three models benchmarked on identical hardware
5	**P4 · Fine-tuning**	Beats the best-prompt baseline on a held-out split
6	**P5 · Real-time**	Latency budget sums to measured total; degrades gracefully

Why P3 comes third, not last. Observability built after the fact is instrumentation bolted onto code that was not designed to be observed. Built at step 3, every subsequent project inherits tracing for free — and you can prove each change improved something.

Why P4 comes late. Fine-tuning is only defensible when you can show a prompted baseline it beats. Without P1's evaluation harness you have no baseline, and the project reduces to "I ran a training script."

Effort shape ESTIMATE

Project	Where the time actually goes
P1 RAG	Data prep and eval-set curation, not the retrieval code
P2 Local SLM	Benchmark methodology and controlling for noise
P3 Observability	Deciding what to measure; wiring is quick
P4 Fine-tuning	Dataset cleaning — typically the dominant cost
P5 Real-time	Failure handling and interruption logic

1 · PRODUCTION RAG

retrieval as an engineered system

Reference architecture

```
# serving path — each stage independently measurable
query → rewrite → [dense ANN] ∪ [BM25] → RRF fuse
      → cross-encoder rerank → parent expansion
      → prompt assembly → LLM → citation validator
```

Why two retrievers then a reranker. Dense and sparse retrieval fail in complementary ways — embeddings blur exact tokens like error codes and SKUs; lexical search misses paraphrase. Fusing them raises recall. The reranker then converts that recall into precision, because a cross-encoder scores query and passage *jointly* and is far more accurate than the bi-encoder used for indexing. MECHANISM

Fusion — RRF

```
score(d) = Σᵢ 1 / (k + rankᵢ(d))    # k = 60 is the
# conventional default (Cormack et al., 2009)
```

Rank-based, so it needs no score normalisation between retrievers whose scales are unbounded and query-dependent.

Configuration to tune, not copy STARTING POINTS

Knob	What it trades
Chunk size / overlap	Retrieval precision vs context sufficiency
Candidates before rerank	Recall ceiling vs rerank latency and cost
Final k after rerank	Evidence coverage vs prompt cost and dilution
Fusion weighting	Lexical vs semantic emphasis for your query mix

Optimal values are properties of your corpus and query distribution. Short factual lookups and long analytical questions want different settings. Sweep them against measured recall — any number quoted without that measurement is someone else's answer to a different problem.

Citation enforcement — make refusal a code path

- Require the model to emit claim → source-chunk references in a parseable form
- **Validate after generation:** every claim must map to a retrieved chunk
- On failure, **refuse or degrade explicitly** rather than returning an ungrounded answer

This is the difference between a demo and a trustworthy system. Anything can generate a plausible answer. A system that reliably declines when evidence is absent is doing engineering the plausible one is not.

Acceptance criteria — phase 3 is done when: a golden set exists with verified question → source-chunk mappings; retrieval recall and faithfulness are measured and recorded as baselines; CI runs the eval on every pull request and fails below threshold; and you can demonstrate the gate blocking a deliberately seeded regression.

2 · LOCAL INFERENCE

the performance model

Memory model — derive it before you download ARITHMETIC

```
# weights
bytes ≈ params × bytes_per_param
  fp16 → 2   int8 → 1   4-bit → ~0.5 (+ scale overhead)

# worked example, 7B parameters
  fp16 : 7e9 × 2   = 14 GB
  4-bit: 7e9 × 0.5 ≈ 3.5 GB + overhead

# KV cache grows with context length
per_token ≈ 2 × layers × kv_heads × head_dim × bytes
```

This is why quantization is the enabling trick for local work. The weight arithmetic is deterministic — you can decide what fits on your hardware before pulling a single model. The KV cache is the term people forget: it scales with context length and can dominate at long contexts.

Two latencies, two different bottlenecks MECHANISM

Metric	What governs it
Time to first token (prefill)	Processes the whole prompt in parallel — **compute-bound**, scales with prompt length
Inter-token latency (decode)	One token at a time, re-reading weights — **memory-bandwidth-bound**, roughly flat per token

Reporting a single "latency" number hides this. A long prompt with a short answer and a short prompt with a long answer are bottlenecked by different hardware resources. Measure TTFT and inter-token latency separately, or your benchmark cannot be reasoned about.

Benchmark methodology — what makes it credible

- **Warm-up runs discarded** — first-call load and cache effects distort everything
- **Fixed prompt set and decode length** across all models
- **Report median and p95**, not the mean — and state the run count
- **Control thermal throttling** on laptops; sustained runs differ from bursts
- **Record the hardware** — chip, RAM, and whether GPU/Metal acceleration was active
- Separate **quality** from **speed**: same prompts, graded independently

An uncontrolled benchmark is worse than none. It invites a reviewer to ask one question you cannot answer, and the whole project's credibility goes with it. Publish the method next to the numbers.

Acceptance criteria: three models benchmarked on identical, documented hardware; TTFT and inter-token latency reported separately with median and p95; memory footprint recorded; output quality graded on a fixed prompt set; and a quantization comparison showing the quality-versus-speed trade with numbers on both axes.

3 · OBSERVABILITY

designing what to measure

Span model — one trace per request

```
request
├─ rewrite            tokens, latency
├─ retrieve.dense     k, ids, scores, ms
├─ retrieve.lexical   k, ids, scores, ms
├─ fuse               merged order
├─ rerank             in→out ids, ms
├─ generate           model, in/out tokens, ms
└─ validate.citations pass/fail, reason
```

Record chunk IDs at every stage, not just the final set. The diagnostic question is almost always "was it retrieved and then lost, or never retrieved at all?" — and only stage-by-stage IDs answer it. This single decision determines whether the system is debuggable.

Conventions and cardinality

- OpenTelemetry publishes **GenAI semantic conventions** (the `gen_ai.*` attribute namespace). Following them means your traces work with standard tooling instead of a bespoke schema EVOLVING SPEC
- **Keep high-cardinality values out of metric labels.** Query text and chunk IDs belong on spans; metrics should carry bounded dimensions like model, tenant tier, and status
- **Tail sampling** where volume demands it — sample successes, retain all errors and slow outliers, since those are what you investigate

The metric set worth defending

Metric	Why this one
Latency p50 / p95	Averages conceal the tail users actually complain about
Cost per request	Tokens × price, attributable by feature and tenant
Citation coverage	Share of answers grounded in retrieved evidence
Refusal rate	Rising = retrieval degrading; falling to zero = enforcement broken
Retrieval score distribution	Leading indicator — drifts before quality visibly drops

Refusal rate is the underrated one. Most teams never track it, yet it moves in both directions for different reasons — and a refusal rate of zero usually means the enforcement path is silently broken rather than that the system is perfect.

Acceptance criteria: any past request can be replayed from its trace; a dashboard shows p50/p95, cost, coverage and refusal over time; CI blocks a merge when eval metrics regress; and prompts and retrieval config are versioned alongside code so a behaviour change is attributable to a commit.

4 · FINE-TUNING

mechanics and proof

What LoRA actually does Hu et al., 2021

```
# base weights frozen; learn a low-rank update
W' = W + ΔW,   ΔW = B·A
     B ∈ ℝ^(d×r),  A ∈ ℝ^(r×k),  r « min(d,k)

# trainable params per adapted matrix
  full : d × k
  LoRA : r × (d + k)      ← orders of magnitude fewer
```

That rank decomposition is the whole idea. Because the base weights stay frozen and only a thin factorised update is learned, memory drops enough to train meaningfully on a single accessible GPU. **QLoRA** (Dettmers et al., 2023) extends this by holding the base model in 4-bit while training the adapters at higher precision.

Why DPO rather than RLHF Rafailov et al., 2023

- Classic RLHF trains a **separate reward model**, then optimises the policy against it with reinforcement learning — multiple models, an unstable loop
- **DPO optimises directly on preference pairs**, removing the separate reward model and the RL stage
- Practical consequence: markedly simpler to run and reproduce, which is why it is tractable as a portfolio project at all

The evaluation that makes the claim credible

Requirement	Why it is non-negotiable
Prompted baseline first	"Improvement" is meaningless without the best the base model does with a well-engineered prompt
Held-out split, deduplicated	Near-duplicates across train and test inflate results silently
Task metrics, not loss	JSON validity, exact match, refusal correctness — loss curves are not results
General-capability check	Detects regression on everything you did not train for

Catastrophic forgetting is the failure mode to name. A model tuned hard on one narrow task can degrade elsewhere. Holding out a small general evaluation and reporting it — even if it moved slightly the wrong way — demonstrates more maturity than a single flattering number.

Data quality dominates data quantity. Inconsistent or badly formatted training examples teach the model to be inconsistent and badly formatted. Budget most of the project for dataset construction and cleaning; that is where the result is actually determined.

Acceptance criteria: a documented prompted baseline; SFT results on a deduplicated held-out split; DPO showing incremental gain over that SFT baseline; task metrics reported before and after; and an honest account of a failed run and what changed.

5 · REAL-TIME VOICE

latency as an architecture

The budget — measure every term separately

```
total ≈ capture + endpointing + ASR
      + LLM TTFT + TTS TTFB + playout
      + transport & queueing overhead
```

Endpointing is the hidden dominant term. Before any model runs, the system must decide the user has stopped speaking. Wait too long and the assistant feels sluggish; too little and it interrupts. Teams optimise the model and never instrument this, then cannot explain where the time went.

Architectural moves that cut perceived latency

- **Stream every stage.** Use partial ASR transcripts, stream LLM tokens, and begin TTS on the first complete sentence rather than the full response — this overlaps stages instead of serialising them
- **Optimise TTFT over total generation.** The user hears the first syllable; what follows streams behind it
- **Support barge-in.** Interruption must cancel in-flight TTS and LLM work — without it the system feels unnatural regardless of raw speed
- **Prefer WebRTC over plain WebSockets for audio transport** where network conditions vary — it is built for real-time media with jitter and loss handling DESIGN INTENT

Why the target is roughly conversational. Research on turn-taking in natural conversation finds inter-speaker gaps typically on the order of a couple of hundred milliseconds (Stivers et al., 2009). You will not hit that with a full ASR-LLM-TTS stack — but it explains why overlapping the stages matters far more than shaving any single one.

Degradation — design the failure paths

Failure	Designed response
ASR unavailable	Fall back to text input; say so explicitly
LLM timeout	Emit a holding response, or a smaller local model
TTS unavailable	Return text; never hang silently
Any stage slow	Hard timeout with a partial result — bounded, never indefinite

Replay mode is the highest-value debugging investment. Recorded audio fed back through the pipeline makes real-time failures reproducible. Without it you are debugging something that only happens live and never twice the same way.

Acceptance criteria: a per-request latency breakdown whose terms sum to measured total; stages overlapped rather than serialised; every external dependency has a timeout and a defined fallback; barge-in cancels in-flight work; and replay reproduces a captured failure.

EVALUATION AS THE SPINE

the discipline all five share

The single strongest signal across the portfolio. Any competent engineer can assemble these pipelines from documentation. What separates candidates is whether they can *prove* the system works and detect when it stops — that is the scarce skill, and it is demonstrated identically in all five projects.

The pattern, applied five times

Project	Baseline	Gate
P1 RAG	Naive top-k retrieval	Recall + faithfulness in CI
P2 Local	Largest model, unquantized	Quality floor at target throughput
P3 Obs	Current metric values	Merge blocked on regression
P4 Tune	Best prompted base model	Held-out task metrics
P5 RT	Serialised, unoptimised pipeline	p95 within budget

Every row has a baseline. A number with nothing to compare against is decoration. "Faithfulness 0.87" says nothing; "0.71 → 0.87 after adding reranking, on the same 150-item set" is an engineering result.

Offline and online are different questions

- **Offline** — fixed dataset, reproducible, gates merges. Answers "did this change break something?"
- **Online** — live traffic, user signals, drift. Answers "is it working for real users now?"
- Offline evaluation cannot detect a shift in what users ask; online cannot block a bad deploy. Both, or neither is sufficient

On LLM-as-judge

Automated grading makes continuous evaluation affordable, and is often the only practical option at scale. But a judge is a model with its own biases — including documented tendencies toward longer or self-similar answers. **Calibrate it against human labels on a sample** and report that agreement rate. Doing so is itself a strong signal: it shows you treat your measuring instrument as something requiring validation.

Failed queries are the roadmap. Every production failure you diagnose should end as a new golden-set case. That is how the harness compounds in value rather than decaying.

REPOSITORY STANDARDS

what a reviewer opens first

```
project/
├─ README.md           results + numbers, not features
├─ ARCHITECTURE.md     diagram + data flow
├─ DECISIONS.md        ADRs: choice, alternatives, why
├─ eval/
│  ├─ golden_set.jsonl
│  ├─ run_eval.py
│  └─ results/         dated, committed
├─ configs/            prompts + retrieval params
├─ src/
└─ .github/workflows/ eval gate on PR
```

The README is the interview

- Open with **the problem and the measured outcome**, not a feature list
- A results table with **baseline → current** on every metric
- **Architecture diagram** — one image beats three paragraphs
- An explicit **limitations** section: what it does badly and why
- How to reproduce the evaluation in one command

Architecture decision records

One short entry per significant choice: the decision, the alternatives considered, and why. Choosing Weaviate over Chroma is unremarkable; **documenting why, and what you would have chosen at ten times the scale, is what a senior reviewer is looking for.** ADRs are cheap to write and are the clearest available evidence of engineering judgment.

Commit the evaluation results. Dated result files in version control show the quality trajectory over time — and prove the harness was actually run rather than merely written.

WHAT FAILS REVIEW

observed anti-patterns

Anti-pattern	What the reviewer concludes
No numbers anywhere	Never measured it; cannot tell if it works
Metrics with no baseline	Does not understand what evidence is
Prompts buried in source	Has not treated prompts as configuration
Five disconnected repos	Followed five tutorials
Only the happy path	Has not operated anything real
Latest model, no rationale	Selects by fashion, not by requirement
Eval set built from outputs	Circular — grades the system against itself
"Works on my machine" benchmarks	No methodology; numbers unusable
Fine-tuned with no baseline	Cannot show tuning helped at all
No limitations section	Either has not looked, or is hiding it

The most common single defect: a polished front end over a pipeline with no evaluation. It reads as someone who optimised for the demo rather than the system — precisely the opposite of the signal intended.

The inverse is disproportionately powerful. A plain interface over a rigorously measured, well-instrumented pipeline reads as an engineer who knows where the difficulty actually lives.

Questions to be ready for

- Why this chunk size — what did you try and measure?
- How do you know retrieval is the bottleneck and not the model?
- What does this cost per thousand requests?
- What breaks first at ten times the traffic?
- What would you do differently starting again?

PORTFOLIO ACCEPTANCE CRITERIA

confirm before you put any of this in front of a hiring manager

- ☐ **Every project reports a baseline** and a current number on the same metric
- ☐ **Golden set is hand-verified** — not generated by the system it evaluates
- ☐ **Retrieval measured separately** from generation quality
- ☐ **CI gate demonstrably blocks** a seeded regression
- ☐ **Chunking parameters justified** by a measured sweep
- ☐ **Reranking impact quantified** before and after on one set
- ☐ **Refusal path exists and is tested** — the system can decline
- ☐ **TTFT and inter-token latency reported separately**
- ☐ **Benchmark methodology published** alongside the numbers
- ☐ **Hardware documented** — chip, memory, acceleration
- ☐ **Quantization trade shown** on both quality and speed axes
- ☐ **Chunk IDs logged at every pipeline stage**
- ☐ **p50 and p95 tracked**, never averages alone
- ☐ **Cost per request quantified** in currency
- ☐ **High-cardinality data kept off metric labels**
- ☐ **Fine-tuning beats a documented prompted baseline**
- ☐ **Held-out split deduplicated** against training data
- ☐ **General-capability regression checked** after tuning
- ☐ **Latency budget terms sum to measured total**
- ☐ **Every external dependency has a timeout and fallback**
- ☐ **Barge-in cancels in-flight work**
- ☐ **Replay reproduces a captured failure**
- ☐ **Prompts and configs under version control**
- ☐ **ADRs record the significant choices**
- ☐ **Limitations section is honest and specific**
- ☐ **Evaluation reproducible in one command**

The through-line, stated once. These five projects are not five demonstrations of five frameworks. They are one demonstration, repeated five times, of a single engineering habit: **establish a baseline, change one thing, measure the delta, and gate on it.** That habit is what production AI work consists of, it is what is scarce in the candidate pool, and it is the only thing here that will still be true when every tool named in these projects has been replaced.